

Casting Defect Detection: Merged Evaluation Report

Binary classification of cast impeller images (**normal** vs **defective**; defects are dots, holes and scratches). Two approaches were built and evaluated:

	Approach A	Approach B
Model	LightCNN , trained from scratch	EfficientNet-B0 , ImageNet transfer learning
Variants	single model (Optuna-tuned)	B1: head only (frozen backbone). B2: head + last 3 blocks fine-tuned
Input	1 channel, 300x300, per-image standardization	3 channels (grayscale replicated), 224x224, ImageNet mean/std

1. Dataset and splits

- 6,000 images (3,000 ok_front, 3,000 def_front), all 300x300 JPEG, grayscale stored as RGB, 0 corrupt files.
- Classes are balanced, so imbalance correction was not needed (pos_weight is about 1.0 in both pipelines).

	A: LightCNN	B: EfficientNet-B0
Split method	Stratified, group-aware (near-duplicates by perceptual hash kept in one split)	Stratified random split (train_test_split)
Train	~71%	4,199 (2,099 normal / 2,100 defective)
Validation	~14%	901 (451 / 450)
Test	864 (429 / 435)	900 (450 / 450)
Threshold chosen on	validation, maximize F2	validation, best precision subject to recall >= 0.98

EDA finding (brightness shortcut). Defective images are darker (mean pixel 139.3 vs 149.8) and lower contrast. Brightness alone separates the classes with AUC about 0.88. Approach A removes this with per-image standardization. Approach B uses ImageNet normalization plus `ColorJitter(brightness=0.2)`, which does not remove the global brightness cue.

2. Models and training

A: LightCNN (scratch). Stride-2 stem, then depthwise-separable conv stages, global max+avg pooling, dropout, 1 logit. Hyperparameters tuned with Optuna (validation average precision). AdamW, OneCycle, mixed precision, early stopping. Augmentation: flips, 90-degree rotations, small rotation and translation (no cutout or tight crops that could erase small defects).

B: EfficientNet-B0 (transfer). Classifier replaced with `Dropout(0.3) -> Linear(1280, 1)`; 4,008,829 parameters in total (about 16 MB in fp32).

- B1, head only:** backbone frozen, 1,281 trainable parameters, AdamW lr 1e-3, early stopped at epoch 11 (best val F1 0.8943).
- B2, fine-tuned:** blocks 6-8 unfrozen (lr 1e-4) plus head (lr 3e-4), 15 epochs max, early stopped by validation AP (best val AP 0.9997). Augmentation: flips, rotation up to 180 degrees (impeller is round), brightness/contrast jitter.

3. Test results

Model	Threshold	Precision (defective)	Recall (defective)	F1	ROC-AUC	TN	FP	FN	TP	Accuracy
A LightCNN (val-tuned)	0.248	0.9977	0.9954	0.9965	0.9989	428	1	2	433	0.9965
A at default threshold	0.500	1.0000	0.9908	0.9954	0.9989	429	0	4	431	0.9954
B1 EfficientNet head-only	0.0635	0.8491	1.0000	0.9184	0.9980	370	80	0	450	0.9111
B2 EfficientNet fine-tuned	0.9653	1.0000	0.9933	0.9967	about 1.0 (prints 1.0000)	450	0	3	447	0.9967

Per-class report, B2: normal precision 0.9934 / recall 1.0000; defective precision 1.0000 / recall 0.9933 (support 450 each).

Per-class report, A: normal precision 0.9953 / recall 0.9977; defective precision 0.9977 / recall 0.9954 (support 429 / 435).

Uncertainty (95% intervals).

- A (bootstrap): precision 0.993-1.000, recall 0.988-1.000, F1 0.992-1.000, AUC 0.997-1.000.
- A (Wilson, from counts): recall 0.983-0.999, specificity 0.987-1.000.
- B2 (Wilson, from counts): recall 0.981-0.998, specificity 0.992-1.000.
- B1 (Wilson, from counts): specificity 0.784-0.855, i.e. roughly 15-22% of good parts falsely rejected.

How to read the comparison

1. **A and B2 are statistically indistinguishable.** They differ by 1-3 images out of about 880 and the intervals overlap heavily. Neither can be called better from these numbers alone.
2. **B1 (head only) is not usable.** The frozen ImageNet features rank the images reasonably (AUC 0.998) but cannot separate them cleanly. Reaching recall 0.98 on validation needed a threshold of 0.064, which cost 80 false alarms in 450 good parts (17.8%). Fine-tuning fixed this, showing that this domain needs adapted features.
3. **The comparison is not apples to apples.** The test sets differ (864 vs 900 images), and B's split is not near-duplicate-aware, so B's numbers could be slightly optimistic if near-duplicates cross splits. For a fair comparison, evaluate both models on the same group-aware test split.
4. **Trade-off.** A is a small custom network (parameter count and latency: fill in from chunk 10) versus B2 at 4.0M parameters (about 16 MB). Report measured CPU and GPU latency for both before choosing.

4. Threshold behavior

- **A:** results are stable for thresholds from about 0.25 to 0.50 (0-1 false alarms, 2-4 misses). Lowering to 0.05 gives 0 misses but 265 false alarms.
- **B2:** the chosen threshold of **0.9653 is very high**, and the 3 misses have probabilities of about 0.89, 0.92 and 0.92, just below it. ROC-AUC is about 1.0, meaning the ranking is nearly perfect and some threshold would likely give no errors on this test set. The validation-selected threshold was slightly too strict. Do not pick a new threshold from the test set. Instead, choose it on validation with a margin rule (for example, the midpoint between the highest-scoring normal and the lowest-scoring defective, or maximize F2), or use cross-validation to stabilize it.
- **B1:** the threshold of 0.0635 shows the probabilities are poorly calibrated.

Expected precision at realistic defect rates (A, same recall and false-alarm rate)

Prevalence	50%	20%	5%	1%
Precision	0.998	0.991	0.957	0.812

Treat these as optimistic: the false-alarm rate rests on a single false positive (A) or zero (B2), so it is very uncertain. Validate on real line data before deployment.

5. Error analysis

- **B2 false negatives (3):** all three are subtle. At display size the defects are not visible to the eye, which suggests very small or low-contrast dots/scratches. Their scores (0.89-0.92) are high, so these are borderline misses and not confident ones. B2 had 0 false positives.
- **B1:** 80 false positives (good parts flagged). The false-positive grid was overwritten in the notebook because the error-analysis cell was run after fine-tuning. Re-run it on the head-only weights if you want the comparison in the README.
- **A (to complete):** 2 false negatives and 1 false positive. Fill in what these look like from the FN/FP grids and errors.csv: size and type of defect, position, and whether the false positive is a texture or a possible label error.
- **Suggested next step:** Grad-CAM on all errors to show where each model looks, and a check of whether misses concentrate on the smallest defects.

6. FastAPI deployment

Project layout:

```
task/
  app/
    __init__.py
    config.py          # paths, image size, limits
    inference.py       # model loading + preprocessing + prediction
    main.py            # FastAPI app, validation, logging, error handling
  models/
    best_head_only.pth
```

Important: the deployed checkpoint is the weakest model

models/best_head_only.pth is the **B1 head-only checkpoint**: threshold 0.0635, 80 false alarms out of 450 good parts, F1 0.918. The notebook's last cell saves the much better best_finetuned.pth (B2: threshold 0.9653, 0 false alarms, 3 misses) to /content/drive/MyDrive/casting/. **Replace the file in models/ with best_finetuned.pth and update config.py**, otherwise the service rejects roughly 1 in 6 good parts.

Checkpoint contract (from the notebook)

The checkpoint is a dict with keys model_state, threshold, img_size (224) and classes (["normal", "defective"]).

```
import torch, torch.nn as nn
from torchvision import models, transforms

def load_model(path, device="cpu"):
    ck = torch.load(path, map_location=device)
    m = models.efficientnet_b0(weights=None)          # no download at inference time
    m.classifier = nn.Sequential(nn.Dropout(0.3), nn.Linear(m.classifier[1].in_features, 1))
    m.load_state_dict(ck["model_state"]); m.eval().to(device)
    return m, float(ck["threshold"]), int(ck["img_size"])

# Must match training exactly: RGB, resize, ToTensor, ImageNet normalization (no augmentation)
tf = transforms.Compose([transforms.Resize((224, 224)), transforms.ToTensor(),
                        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])
```

Confidence score: do not use $1 - p$ with a non-0.5 threshold

With thresholds of 0.0635 or 0.9653, a naive "confidence of the predicted class" is misleading (for example, $p = 0.95$ with threshold 0.9653 would be reported as normal with confidence 0.05). Return the raw defect probability and a confidence defined relative to the threshold:

```
def decide(p_defect: float, thr: float):
    label = "defective" if p_defect >= thr else "normal"
    if p_defect >= thr: # map [thr, 1] -> [0.5, 1]
        conf = 0.5 + 0.5 * (p_defect - thr) / (1 - thr)
    else: # map [0, thr] -> (0.5, 1]
        conf = 0.5 + 0.5 * (thr - p_defect) / thr
    return label, round(conf, 4), round(p_defect, 4)
```

Suggested response body: {"predicted_class": ..., "confidence": ..., "defect_probability": ..., "threshold": ..., "model_version": ..., "latency_ms": ...}.

Production checklist for main.py

- Validate content type and file size; reject corrupt or non-image files with a 4xx error; handle decode failures without a 500.
- Load the model once at startup (FastAPI lifespan), not per request; run inference with `torch.inference_mode()`.
- Add /health and /model-info (version, threshold, input size).
- Structured logging of request id, latency and predicted class (never raw image bytes).
- Run CPU inference in a threadpool (def endpoint or `run_in_threadpool`) so the event loop is not blocked.
- Dockerfile with CPU-only PyTorch, pinned requirements.txt, a non-root user, and a few tests (valid image, corrupt file, wrong type, oversize).
- Optional: export to ONNX and compare latency.

Note: I have not seen `app/*.py`, so the points above are a checklist to verify against your code, not a description of it.

7. Known limitations

- **Brightness shortcut:** defective captures are systematically darker. Models may partly learn exposure instead of defects (B is more exposed to this than A). Recommend a brightness-normalization ablation and confirming consistent lighting with the client.
- **Easy dataset, tiny error counts:** 1-3 errors per test set give wide uncertainty; model differences of a few images are not significant.
- **Split differences** between A and B (group-aware vs random) limit direct comparison.
- **Threshold fragility:** B2's very high threshold sits just above its misses; calibrate (temperature scaling) and choose thresholds with a margin.
- **Prevalence:** real lines have far fewer defects; precision will drop and must be validated with realistic data.
- **Scope:** only dots, holes and scratches on one part type and camera setup; no testing under distribution shift.

8. Recommended final configuration

1. Serve **B2 (fine-tuned)** or **A (LightCNN)** depending on measured latency and size on the target hardware. Accuracy is equivalent within noise. Do not serve B1.
2. Re-evaluate the chosen model, and any alternative, on one shared group-aware test split.
3. Add a "needs review" band for scores near the threshold rather than a hard cut.
4. Monitor false-alarm rate and score distribution after deployment.